

Problem Solving Using Pattern Recognition

3. Dimensionality Reduction

Topics

- What is Dimensionality?
- Problems with High Dimension
- Approaches to Dimensionality Reduction
- Feature Selection Methods
- Introduction to Principal Component Analysis method
- Introduction to Linear Discriminant Analysis method
- Hands on workshop

Copyright National University of Singapore

What is Dimensionality?

The **dimension** of a dataset is given as ***n***-rows by ***p***-columns.

- Healthcare data: 600x5; Height, Weight, Blood Pressure, Glucose, Cholesterol
- Netflix rating: 18kx480k; Films x Customer ratings in 2006
- Image data: 800x800x4 image = 800x3200 (4 channels – RGB, Alpha)

- We are usually more concerned about ***p***, the number of features
  - Data with a *small* ***p*** is referred to as **low dimensional data**
  - Data with a *large* ***p*** is referred to as **high dimensional data**

	Anne	Ben	Charlie	Doug	Eve	...
Star Wars	2	5	4	4	3	...
Harry Potter	3	4	5	3	?	...
Pretty Woman	4	?	2	?	5	...
Titanic	5	?	2	1	3	...
Lord of the Rings	?	5	5	4	4	...
⋮	⋮	⋮	⋮	⋮	⋮	⋱



Copyright National University of Singapore

Problems with High Dimensionality

1. Computational & Memory Cost

Consumes large amounts of computational memory and is highly expensive to train, leading to significantly longer training times.

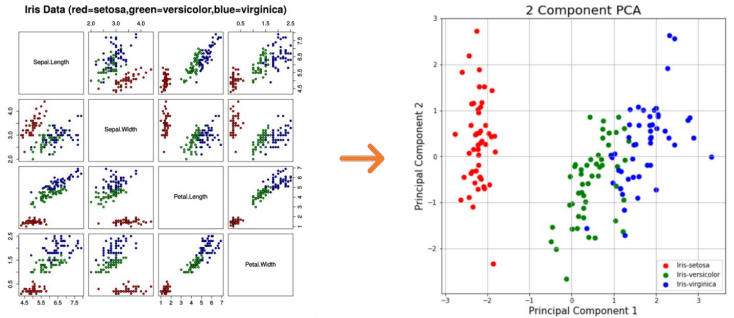
2. Redundancy & Overfitting

Many features contribute little or duplicate information, increasing model complexity, reducing interpretability, and fitting noise rather than signal.

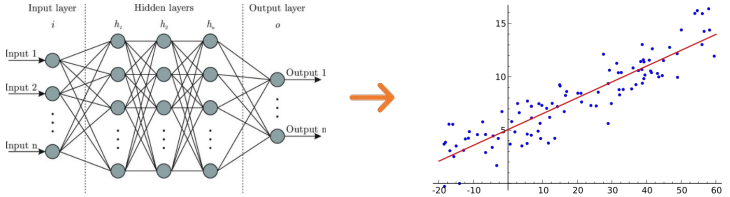
3. Curse of Dimensionality & Visualization

High-dimensional spaces make data visualization beyond 3D extremely difficult, and distances become indistinguishable, causing similarity-based methods to fail.

VISUALIZATION COMPARISON (IRIS DATA → 2D PCA)



REPRESENTATION (NETWORK → PROJECTION)



Copyright National University of Singapore

Curse of Dimensionality

01. Core Concept

As dimensionality grows, data points become **increasingly sparse** in the space they occupy, making traditional data analysis significantly harder.

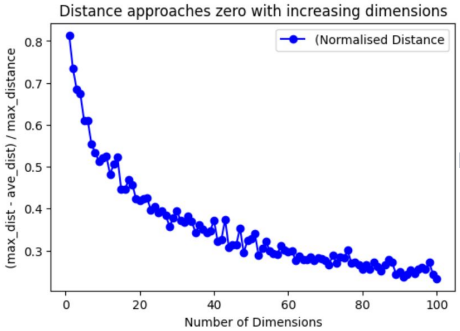
02. Impact on Classification

High dimensionality results in **insufficient data** to build models that generalize well, directly leading to **overfitting**.

03. Impact on Clustering

Distance metrics **lose discriminative power**. Distance-based algorithms like k-NN struggle as distances tend to converge.

Feature 1	Feature 2	Feature 3	Feature 4	Feature 5	Target Label
0.2	0.8	0.1	0.3	0.0	1
0.0	0.5	0.0	0.7	0.4	0
0.9	0.0	0.4	0.0	0.2	1
0.3	0.0	0.6	0.1	0.8	0
0.0	0.2	0.0	0.4	0.6	1



$$\lim_{d \to \infty} \frac{\text{dist}_{\max} - \text{dist}_{\min}}{\text{dist}_{\min}} \to 0$$

Beyer et al. (1999)

Simulation Setup:

500 points randomly generated from 1-100 dimensions. Metric: (max_dist – min_dist) / min_dist

Source: Tan, Steinbach & Kumar, Introduction to Data Mining (2nd Ed), Addison-Wesley

Copyright National University of Singapore

Lower Dimensionality is Desirable



Better Performance

Algorithms work better because reducing dimensions eliminates irrelevant features and noise.



Simpler Models

Fewer attributes lead to a model that is much easier to understand and interpret.



Easy Visualization

Enables effective visualization by reducing the combinations of attributes to look at.



Resource Efficiency

Significantly reduces both the processing execution time and the memory footprint.

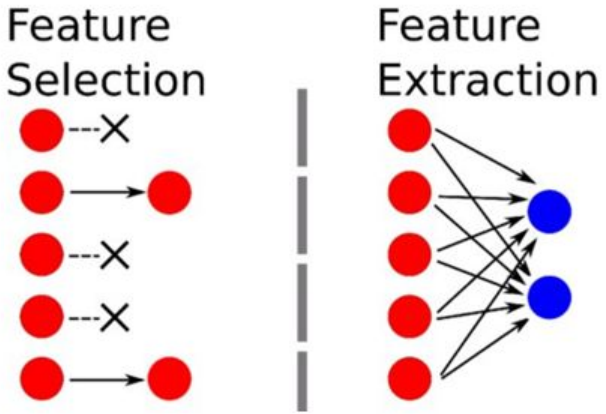
Source: Tan, Steinbach & Kumar, Introduction to Data Mining (2nd Ed), Addison-Wesley

Copyright National University of Singapore

Approaches to Reducing Data Dimensions

There are two different approaches to reducing data dimensionality.

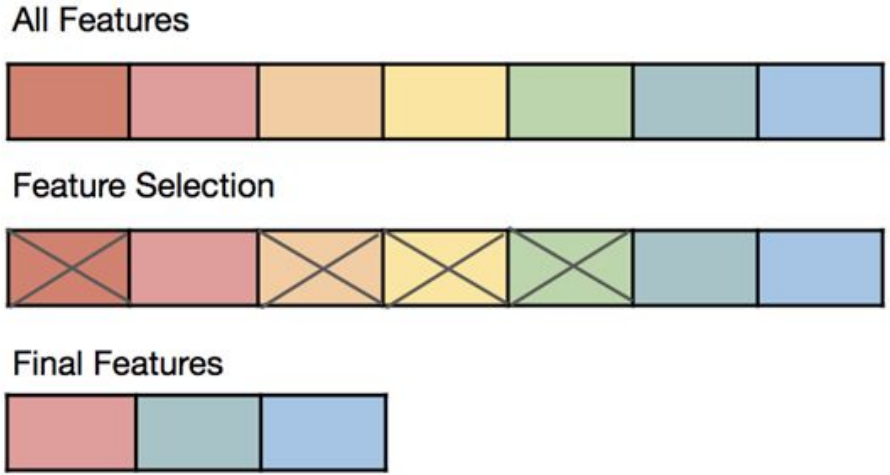
- **Feature Subset Selection Methods**: Remove all features that do not bring much information or add no new information.
- **Feature Extraction Methods**: Keep all original features by extracting them to form new independent variables.



Copyright National University of Singapore

Feature Subset Selection

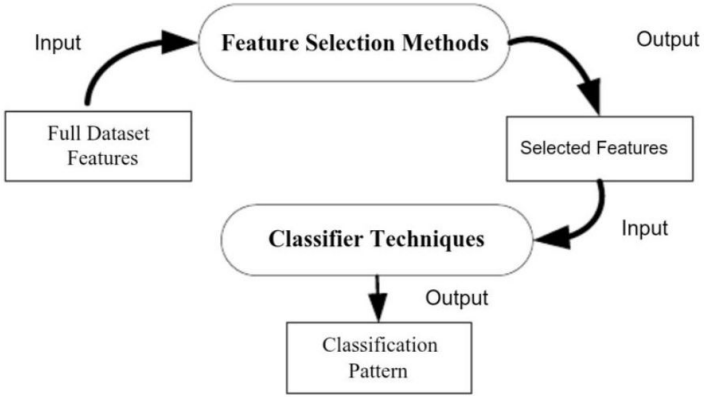
- Feature subset selection is about selecting a subset of features out of the original features in order to **reduce model complexity**, **enhance the computational efficiency** of the models and **reduce generalization error** introduced due to noise by irrelevant features.



Copyright National University of Singapore

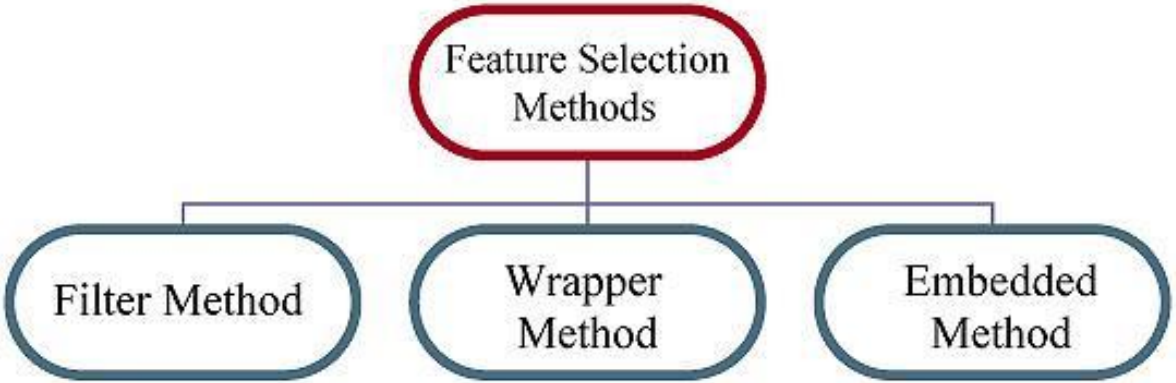
Why use Feature Subset Selection?

- The goal of Feature Subset Selection is to find the best subset of features that does not contain **redundant and irrelevant features** (i.e. less noise)
- The best subset of features helps to:
  - **Improve accuracy** – less misleading data means modelling accuracy improves.
  - **Reduced overfitting-** Less redundant data means less opportunity to make decisions based on noise.
  - **Reduces training time** - Less data means that algorithms train faster.



Copyright National University of Singapore

Feature Subset Selection Methods



Copyright National University of Singapore

Filter Methods

- Filter methods select features from a dataset based on the **intrinsic characteristics** of the features.
- **Statistical techniques** are often used to evaluate the relationship between a predictor and the target variable.
- These methods are simple yet powerful to quickly remove features and are generally the first method used for feature selection.
- Filtering is **least computationally** expensive compared to other methods.



https://en.wikipedia.org/wiki/Feature_selection

Copyright National University of Singapore

Filter Techniques

Pearson's Correlation

Remove *numerical* features that are highly correlated (e.g., r > 0.9) to eliminate redundancy.

Mutual Information

Measures dependency between a feature and the target variable. Works with both categorical and continuous data. Higher scores indicate strong candidates for selection.

Variance Threshold

Remove features that have low variability, or cases where 80% or more of the values are completely identical.

ANOVA (f-test)

Assesses whether feature means differ significantly across target classes. Best for numerical inputs and categorical targets. A p-value < 0.05 suggests a strong visual impact.

https://scikit-learn.org/stable/modules/feature_selection.html

Copyright National University of Singapore

Comparison and Workflow

1. VARIANCE THRESHOLD



1. VARIANCE THRESHOLD

- Unsupervised, removing constant or low-variance features.
- Ignores the target.
- **PROS:** Extremely fast, minimal overhead.
- **CONS:** Ignores target variable, may lose information, scale sensitive.
- **WHEN TO USE IT:** Foundational cleaning for large datasets.

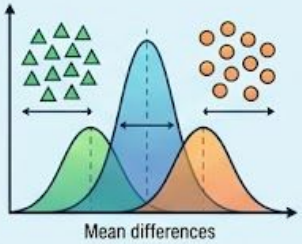
2. PEARSON'S CORRELATION



2. PEARSON'S CORRELATION

- Statistical measure of linear relationship strength (-1 to +1).
- **PROS:** Interpretable score, finds direction, detects collinearity.
- **CONS:** Continuous features/targets only, linear patterns only, outlier sensitive.
- **WHEN TO USE IT:** Regression problems, continuous variables, reducing redundancy.

3. ANOVA (F-TEST)



3. ANOVA (F-TEST)

- Check significant mean differences across categorical target classes.
- **PROS:** Very fast, leverages p-values for selection thresholds.
- **CONS:** Parametric assumptions (normality, equal variance), linear differences primarily.
- **WHEN TO USE IT:** Classification problems with categorical target and continuous features.

4. MUTUAL INFORMATION (MI)



4. MUTUAL INFORMATION (MI)

- Information theory, measuring dependence, reducing uncertainty.
- **PROS:** Handles non-linear and linear dependencies, robust, distribution agnostic.
- **CONS:** Computationally expensive, raw score abstract.
- **WHEN TO USE IT:** Complex non-linear relationships, violated normality assumptions, high computational budget.

GENERAL WORKFLOW RECAP

Apply Variance Threshold First
Eliminate constant features

→

Use Pearson's / ANOVA
Faster linear screen

→

Upgrade to Mutual Information
Complex, non-linear data

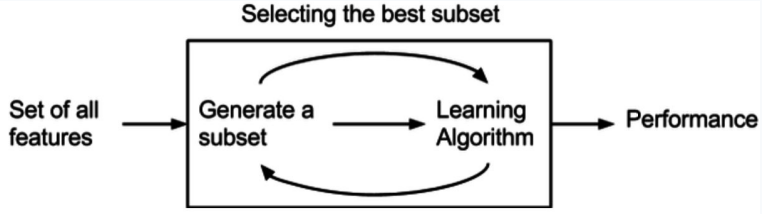
Copyright National University of Singapore

Wrapper Method

Core Concept:

Wrapper methods work by systematically evaluating a subset of features using a **machine learning algorithm in an iterative loop**.

Iterative Selection Process



Source: https://en.wikipedia.org/wiki/Feature_selection

Method Characteristics

- **Performance-Based Selection**

  Selects the "best" feature subset based on the model's performance evaluation criteria.

- **Greedy Search Strategy**

  Finds optimal feature combinations for peak machine learning performance.

- **Computationally Expensive**

  Can be highly demanding and impractical for very large datasets.

Better Accuracy

Usually results in superior predictive accuracy compared to filter methods.

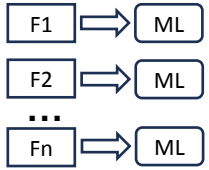
Copyright National University of Singapore

Forward Feature Selection

Forward Feature Selection

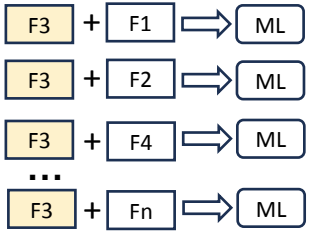
- A greedy search that iteratively finds the best new feature to add to the set of selected features.
- Start initially with zero feature and find the one feature that maximizes a cross-validated score when an estimator is trained on this single feature. Once that first feature is selected, we repeat the procedure by adding a new feature to the set of selected features.
- The procedure stops when the desired number of selected features is reached, as determined by the *n_features_to_select* parameter.

(1) Start with 1 feature and apply the ML model to predict Y.



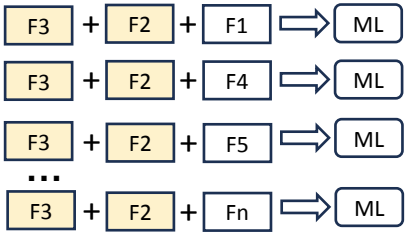
Select the best feature (F3) with highest prediction

(2) Add one feature at a time to the subset and apply the ML model



Select the best feature (F2) and add to subset

(3) Repeat (2) until you reach *n_features_to_select*



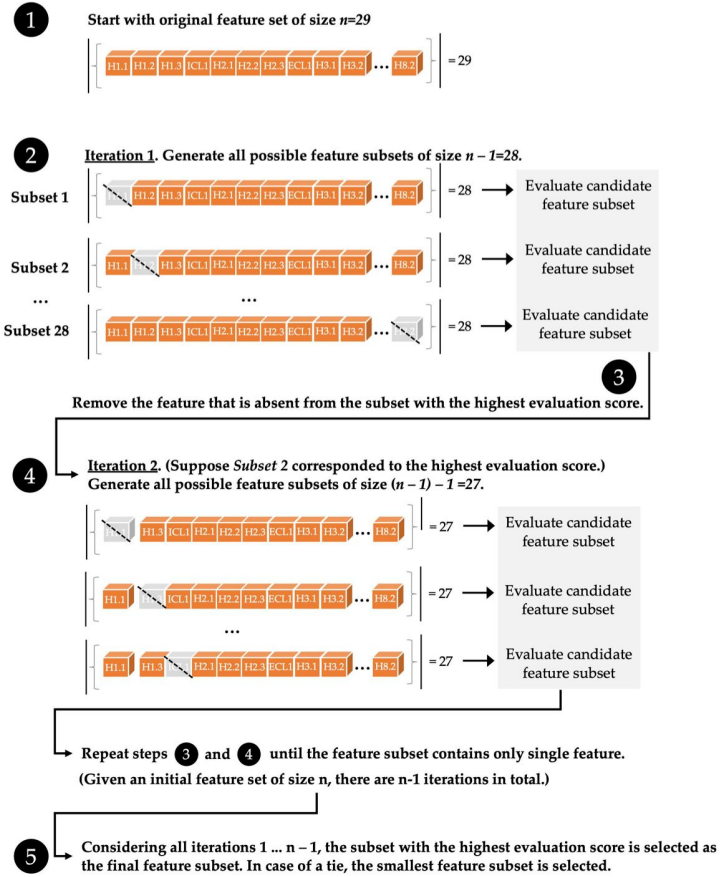
https://scikit-learn.org/stable/modules/feature_selection.html#feature-selection-using-selectfrommodel

Copyright National University of Singapore

Backward Feature Selection

Backward Feature Selection

- Follows the same idea but works in the opposite direction: instead of starting with no feature and greedily adding features, we start with all the features and greedily remove features from the set.
- The subset with the highest score (e.g. accuracy, f1) will be selected. That means one of the weakest feature will be eliminated.



Joe Bemister-Buffington, Alex J. Wolf, Sebastian Raschka, and Leslie A. Kuhn (2020) Machine Learning to Identify Flexibility Signatures of Class A GPCR Inhibition Biomolecules 2020, 10, 454.

Copyright National University of Singapore

Feature Selection Python Example

- In general, forward and backward selection do not yield equivalent results.
- Also, one may be much faster than the other depending on the requested number of selected features: if we have 10 features and ask for 7 selected features, forward selection will need to perform 7 iterations while backward selection would only need to perform 3.

```
ffs = SFS(estimator=logreg, n_features_to_select=10,
          direction='forward', scoring='accuracy')
ffs.fit(X_train, y_train)
print('FFS:'f'{features[ffs.get_support()]}')
```

```
FFS:                        0
1              texture_mean
4           smoothness_mean
5          compactness_mean
6            concavity_mean
7       concave points_mean
8             symmetry_mean
9    fractal_dimension_mean
13                 area_se
21           texture_worst
22         perimeter_worst
```

```
bfe = SFS(estimator=logreg, n_features_to_select=10,
          direction='backward', scoring='accuracy')
bfe.fit(X_train, y_train)
print('BFE:'f'{features[bfe.get_support()]}')
```

```
BFE:                        0
2            perimeter_mean
3                 area_mean
10                radius_se
15           compactness_se
21            texture_worst
22          perimeter_worst
24          smoothness_worst
25         compactness_worst
26           concavity_worst
29  fractal_dimension_worst
```

Copyright National University of Singapore

Recursive Feature Elimination (RFE)

Core Concept

RFE is a powerful feature selection technique that iteratively selects the optimal subset of features using a supervised learning estimator.

How Feature Importance is Obtained:

- **coef_** for linear models
- **feature_importances_** for tree-based models

The algorithm recursively removes the least important features, training the model again at each iteration to adapt to the pruned feature space.

1. **Train Initial Model**
   Train the estimator on the full set of features.
2. **Extract Feature Importance**
   Compute weights or importance rankings.
3. **Prune Weakest Features**
   Remove the least important feature(s).
4. **Re-train on Subset**
   Train model on the newly pruned feature set.
5. **Recursive Loop**
   Repeat Steps 2-4 until target feature count is met.

Reference: https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.RFE.html

Copyright National University of Singapore

Recursive Feature Elimination Python Example

```
from sklearn.feature_selection import RFE
rfe = RFE(estimator=logreg, n_features_to_select=10, step=1)
rfe.fit(X_train, y_train)
print('RFE:'f'{features[rfe.get_support()]}')
```

```
RFE:                    0
0          radius_mean
1         texture_mean
2       perimeter_mean
11          texture_se
13             area_se
20        radius_worst
21       texture_worst
22     perimeter_worst
25   compactness_worst
26     concavity_worst
```

Copyright National University of Singapore

BFE vs RFE

Aspect	Backward Feature Selection	Recursive Feature Elimination
Evaluation for removal	Explicit performance metric (e.g., accuracy, p-value)	Ranks features by importance (e.g., using coefficients or feature importances)
Removal process	Tries each feature and drops the ‘worst’ based on a performance metric.	Drops the least important feature(s) according to the ranking
Computational cost	Higher (tests every possible removal step)	Lower (uses model’s ranking at each step)
Flexibility	Any model, any scoring metric	Requires a model that can rank features
Nature	More statistical / test-based	More machine-learning / model-based

Copyright National University of Singapore

Embedded Method

What is it?

Embedded methods perform feature selection **within the machine learning algorithm itself** during model training, integrating feature selection directly into the optimization objective.

How It Works

01
Train Full Model
Train a machine learning model using the complete set of input features.

02
Derive Feature Importance
Extract importance metrics via coefficients, impurity reduction, or regularization penalties.

03
Filter Non-Important Features
Remove low-scoring features based on the derived importance values.

Advantages

Over Filter & Wrapper Methods

Feature Interactions
Captures complex feature relationships similarly to wrapper methods.

Computationally Efficient
Typically faster than wrappers and performance-comparable to filters.

Reduces Overfitting
Leverages built-in regularization mechanisms like Lasso and Ridge.

Model-Specific Selection
Finds the optimal feature subset tailored precisely to the model being trained.

Source: https://en.wikipedia.org/wiki/Feature_selection

Copyright National University of Singapore

Two ways to perform Embedded feature selection

There are **two main ways** to perform embedded feature selection in Python:

1. Use the Model Directly

Train a model that inherently computes feature importance (e.g., Random Forest, Lasso, Ridge, Gradient Boosting).

- Access **feature_importances_** or **.coef_**
- Manually select features based on those scores

Full Control: Gives you maximum flexibility over how you interpret and apply the importance values.

2. Use SelectFromModel()

Automate the selection process using scikit-learn's built-in meta-transformer wrapper.

- Pass a trained model to **SelectFromModel()**
- Selects features based on a threshold (default or custom)
- Get a reduced feature set without manual inspection

Convenient & Consistent: Ideal for seamless integration into machine learning pipelines.

Copyright National University of Singapore

Embedded Techniques: Classification vs Regression

- For categorical target (Classification)
  - **Logistic Regression**: Select features based on model coefficient and p-values
  - **Decision Trees**: Select features based on purity measure (entropy, Gini)
  - **Random Forests**: Select features using ensemble trees (ranked features)
  - **Gradient Boosting (e.g., XGBoost, CatBoost, LightGBM)**: Provide built-in feature importance scores
- For numerical target (Regression)
  - **Lasso regression** (L1 regularization): Adds penalty proportional to the absolute value of coefficients, often shrinking some to zero.
    https://www.youtube.com/watch?v=NGf0voTMlcs
  - **Ridge regression** (L2 regularization): Adds penalty proportional to the square of coefficients, reducing magnitude but retaining all features.
    https://www.youtube.com/watch?v=Q81RR3yKn30&t=969s

Copyright National University of Singapore

1. Embedded Technique Directly

```
from sklearn.ensemble import RandomForestClassifier

randomforest = RandomForestClassifier(n_estimators=1000, max_depth=5, max_features=8, max_leaf_nodes=8, random_state=43)
rfc = randomforest.fit(X_train,y_train)

print('RFC Training:', rfc.score(X_train, y_train))
print('    Testing :', rfc.score(X_test, y_test))

importances = rfc.feature_importances_

# Sort the feature importances and store into dataframe
feature_importance_df = pd.DataFrame({
    'feature': X_train.columns,
    'Importance': importances}).sort_values(by='Importance', ascending=False)
print(feature_importance_df)
```

```
RFC Training: 0.9874371859296482
    Testing : 0.9766081871345029
                    feature  Importance
27    concave points_worst    0.201827
22         perimeter_worst    0.147077
23              area_worst    0.127753
7       concave points_mean    0.124542
20            radius_worst    0.111958
6           concavity_mean    0.058822
3                area_mean    0.033789
2          perimeter_mean    0.030132
26         concavity_worst    0.027726
13               area_se    0.021583
21          texture_worst    0.019585
0            radius_mean    0.016077
1           texture_mean    0.013129
24        smoothness_worst    0.008914
25       compactness_worst    0.006824
10             radius_se    0.005815
12          perimeter_se    0.005593
28        symmetry_worst    0.005242
29  fractal_dimension_worst    0.004682
4         smoothness_mean    0.004275
16          concavity_se    0.003459
5         compactness_mean    0.003213
19    fractal_dimension_se    0.002643
14          smoothness_se    0.002637
17       concave points_se    0.002400
11             texture_se    0.002312
18           symmetry_se    0.002221
8          symmetry_mean    0.001982
9   fractal_dimension_mean    0.001923
15          compactness_se    0.001864
```

Manually select top 8 features, transform the data into the reduced features and train a RandomForestClassifier model

```
top_features = feature_importance_df['feature'].head(8).tolist()

X_train_selected = X_train[top_features]
X_test_selected = X_test[top_features]
print(X_train_selected.shape)

rfc = randomforest.fit(X_train_selected,y_train)
print('RFCtop8 Training:', rfc.score(X_train_selected, y_train))
print('        Testing :', rfc.score(X_test_selected, y_test))
```

```
(398, 8)
RFCtop8 Training: 0.9748743718592965
        Testing : 0.9766081871345029
```

Copyright National University of Singapore

2. Embedded Technique with SelectFromModel()

```
from sklearn.feature_selection import SelectFromModel

sfm_logreg = SelectFromModel(logreg).fit(X_train, y_train)

sfm_logreg.get_support()
selected_features = X_train.columns[(sfm_logreg.get_support())]
print('Important Features selected by SelectFromModel(RF):\n',pd.DataFrame(selected_features))
print('\nNumber of Feaures (Not Ranked) selected:', len(selected_features))
```

```
Important Features selected by SelectFromModel(RF):
                  0
0       radius_mean
1    perimeter_mean
2           area_se
3      radius_worst
4     texture_worst
5   perimeter_worst
6  compactness_worst
7    concavity_worst

Number of Feaures (Not Ranked) selected: 8
```

After auto feature selection, transform the data into the reduced features and train a logistic regression model

```
X_train_new = sfm_logreg.transform(X_train)
X_test_new = sfm_logreg.transform(X_test)

# run the logreg model with SFM selected features
sfm_lr = logreg.fit(X_train_new, y_train)
print('SFMLR Training:', sfm_lr.score(X_train_new, y_train))
print('      Testing :', sfm_lr.score(X_test_new, y_test))
```

```
SFMLR Training: 0.9447236180904522
      Testing : 0.9590643274853801
```

Copyright National University of Singapore

Feature Selection methods Summary

- **Filter Method**
  - Features are selected before the application of machine learning.
  - Features are selected independent of the analytical task using univariate analysis
- **Wrapper Method**
  - Similar to a greedy search, it evaluates all the possible combinations of features using a learning algorithm's performance as evaluation criteria
  - Retains the feature subset that produces the best results
  - Need to specify number of features to select: ***n_features_to_select***
- **Embedded Method**
  - Feature selection occurs intrinsically as part of ML model building.
  - Specifically, the algorithm itself decides which attributes to use and which to ignore in order to maximize its performance
  - Can use the ***select_from_model_*** or directly running the Model itself

Copyright National University of Singapore

Feature Selection Methods Comparisons

	Filter	Wrapper	Embedded
How it works	Uses univariate, generic algorithm-independent criteria	Uses the machine learning algorithm's performance criteria to find optimal features	Feature selection occurs during model training, often through regularization or model-specific criteria
Computation Speed	Fast	Very High Computation time with many features (except for RFE)	Somewhere between Filter and Wrapper methods
Overfitting problem	Avoids overfitting but sometimes may fail to select best features	May overfit because it involves training of models with different feature combinations	Helps reduce overfitting through some penalty mechanism
Prediction Performance	May not yield the best performance.	Often best performance but comes at a high cost	Somewhere between Filter and Wrapper methods.
Techniques	Correlation, Variance Threshold, Mutual Information, ChiSquare, Anova	Forward Selection, Backward Elimination, Recursive Feature Elimination	Lasso, Ridge, Randomforests, logistic regression

Copyright National University of Singapore

PRINCIPAL COMPONENT ANALYSIS

Feature Extraction Method

Copyright National University of Singapore

Introduction to Principal Component Analysis

Principal Component Analysis (PCA) is a powerful statistical technique for **dimensionality reduction** that transforms a large set of variables into a smaller set of uncorrelated components while retaining most of the original information.

Principal Components

New variables formed as linear combinations of the originals, designed to be uncorrelated.

Variance Maximization

PCA finds directions (components) along which the data varies the most to retain maximum info.

Orthogonality

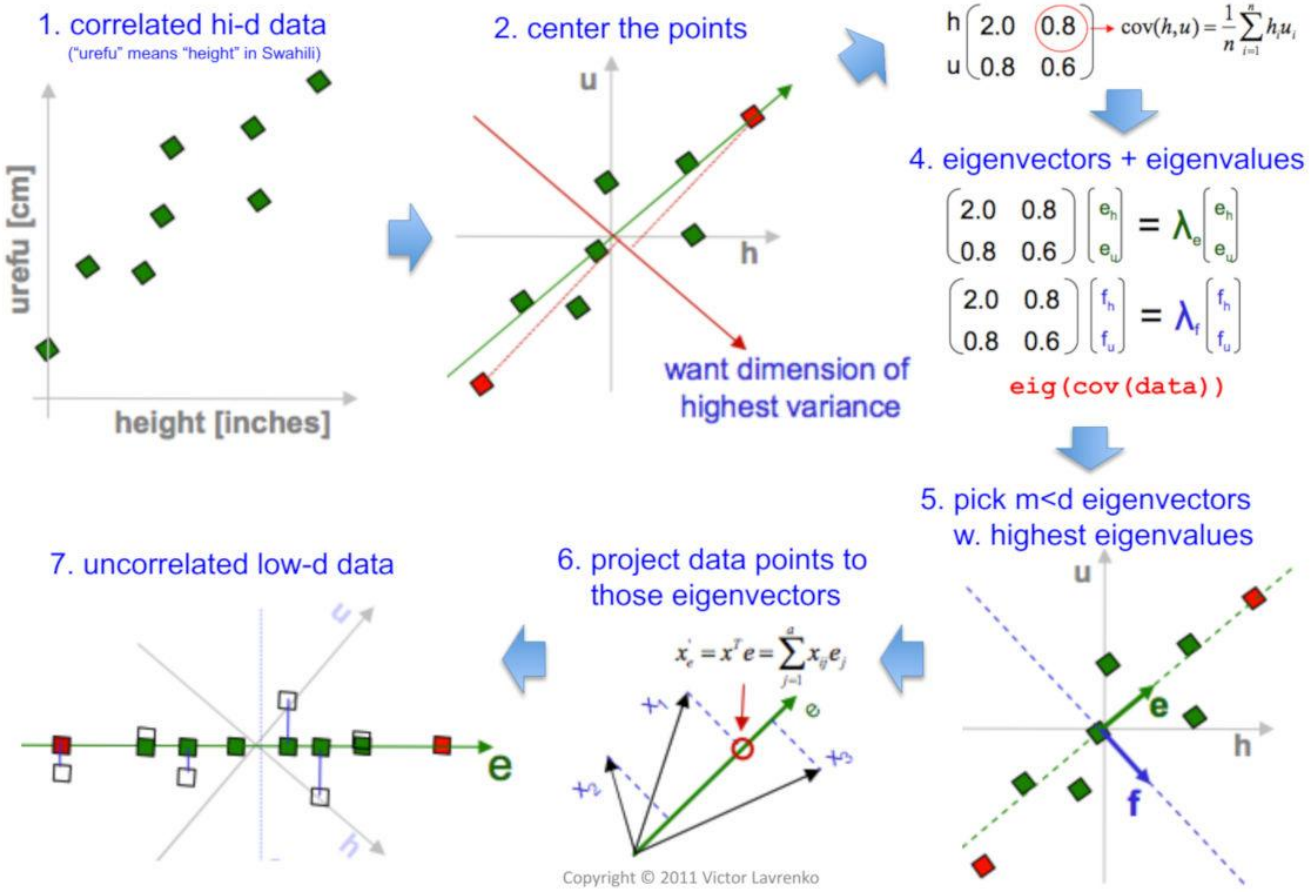
Components are at right angles to each other, completely eliminating data redundancy.

Pedagogical Focus Note

PCA can be mathematically complex—even for quantitatively strong students. We will focus on building an **intuitive understanding** of PCA and explore its applications using packaged libraries such as **scikit-learn**.

Copyright National University of Singapore

PCA in a nutshell



PCA Intuition

Dataset with 8 features
(Usually features are correlated)

Income	Education	Age	Residence	Employ	Savings	Debt	Credit cards
50000	16	28	2	2	5000	1200	2
72000	18	35	10	8	12000	5400	4
61000	18	36	6	5	15000	1000	2
88000	20	35	4	4	980	1100	4
91100	18	38	8	9	20000	0	1
45100	14	41	15	14	3900	22000	4
36200	14	29	6	5	100	7000	5
41000	12	34	9	8	5000	200	3
40000	16	32	8	7	19000	1760	2
32000	16	30	2	2	16000	550	1
29000	16	28	1	4	2100	4600	2
21240	12	26	2	2	100	10010	3
58700	12	38	9	9	4500	7800	5
41000	14	29	5	4	300	10000	6
38720	16	36	11	11	24500	540	2
88240	16	38	13	12	13600	8100	2
40000	18	39	7	6	16000	1300	2

Using Linear Algebra, find a linear combination of all the features such that it captures as much of the original information as possible

PC1 = 0.314*Income +
0.237*Education +
0.484*Age +
0.466*Residence +
0.459*Employ +
0.404*Savings +
-0.67*Debt +
-0.123*Credit cards

We hope that we can use fewer features to represent the original data

	PC1	PC2	PC3
0	-1.36	-1.42	0.17
1	1.79	0.31	1.51
2	1.25	-1.44	0.33
3	0.60	-1.23	3.17
4	3.09	-1.76	0.66
5	2.26	4.58	0.05
6	-1.47	1.59	0.53
7	0.19	0.93	-0.92
8	0.91	-0.68	-1.11
9	-0.82	-1.95	-1.37
10	-1.77	-0.68	-0.47
11	-2.97	1.13	-0.93
12	0.72	2.63	0.60
13	-1.76	2.11	1.18
14	2.46	-0.28	-1.62
15	3.27	0.78	0.74
16	1.50	-1.02	-0.54
17	3.67	0.23	-1.91
18	-2.91	1.84	0.14
19	-1.26	-1.45	0.53
20	-1.75	-0.42	-0.13

High Dimension → PCA Projection → Lower Dimension

Copyright National University of Singapore

What are Principal Components?

- Principal Components are new variables created through PCA. They are designed to **capture the most important patterns** in the data by maximizing variance and reducing redundancy.
- Key Characteristics
  - **Linear Combinations**: Each principal component is a weighted sum of the original features.
  - **Uncorrelated**: Principal components are statistically independent from one another.
  - **Ordered by Importance**: The first component captures the most variance, the second captures the next most, and so on.
  - **Orthogonal**: Components are at right angles to each other—no overlap in the information they carry.

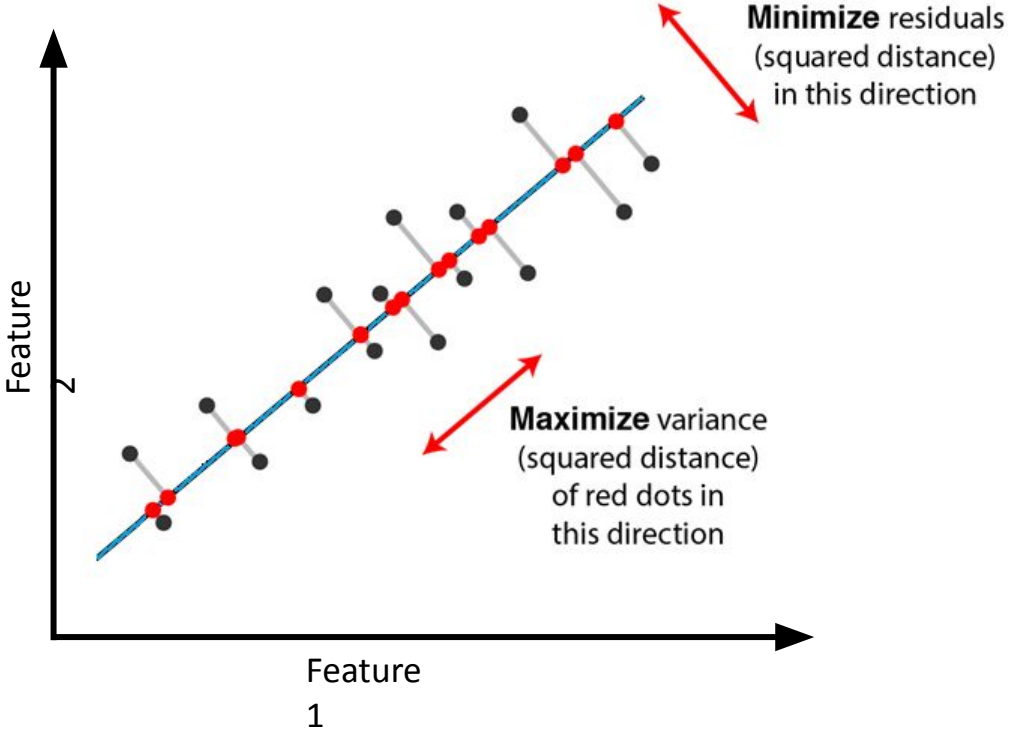
Each component is a weighted sum of the x_i where the a_{ij} are the weights, or coefficients.

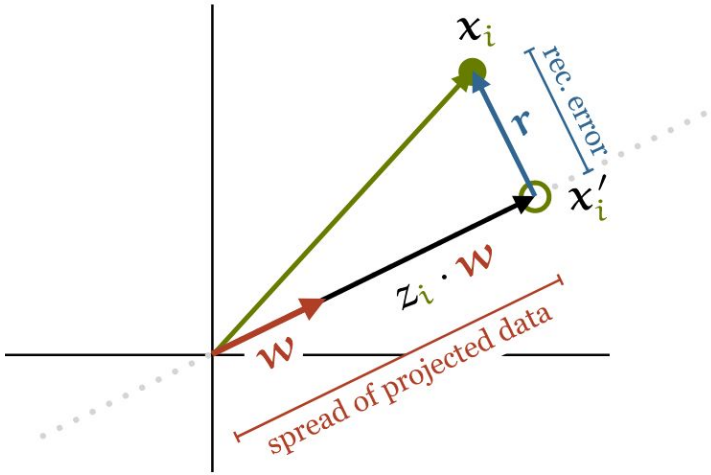
$$
\begin{aligned}
y_1 &= a_{11}x_1 + a_{21}x_2 + \cdots + a_{p1}x_p \\
y_2 &= a_{12}x_1 + a_{22}x_2 + \cdots + a_{p2}x_p \\
&\vdots \\
y_p &= a_{1p}x_1 + a_{2p}x_2 + \cdots + a_{pp}x_p.
\end{aligned}
$$

y_s principal component
x_s original Feature
a_{ij} coefficients

Copyright National University of Singapore

Principal Component Analysis Method

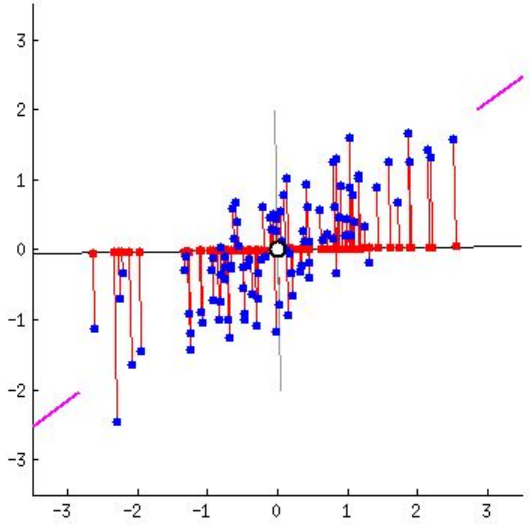




Copyright National University of Singapore

Principal Component Analysis Method

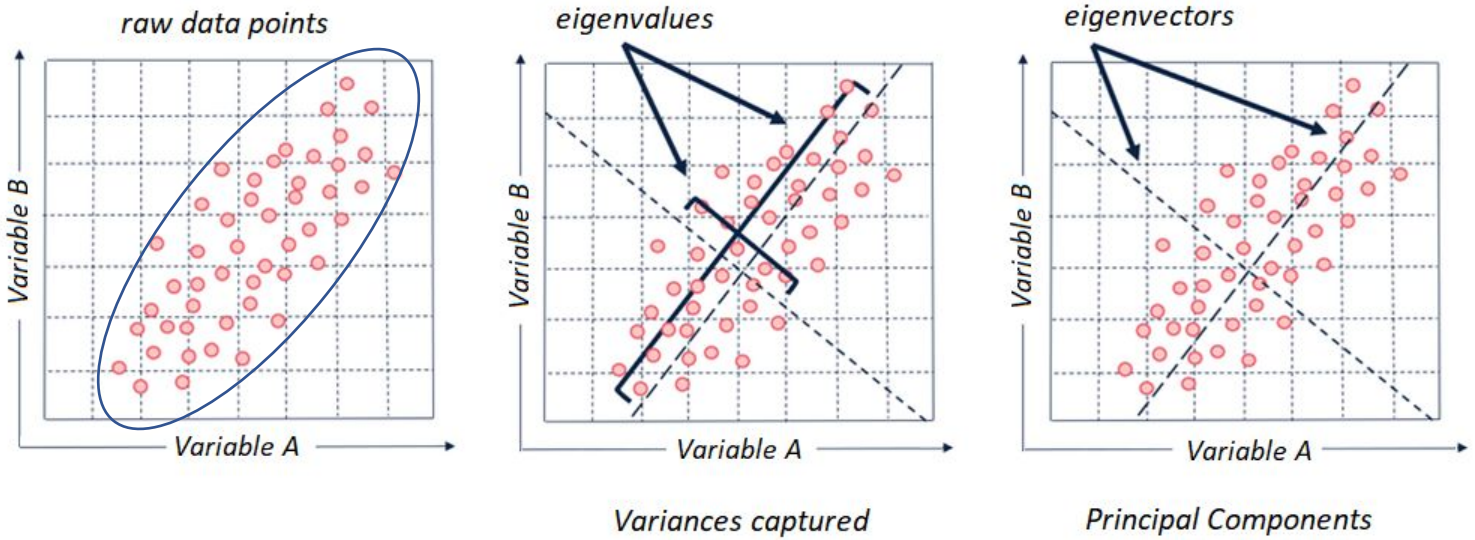
- PCA tries to find a new axis that captures the **maximum variance** within the data once it is projected onto the new axis:



https://i.stack.imgur.com/lNHqt.gif

Copyright National University of Singapore

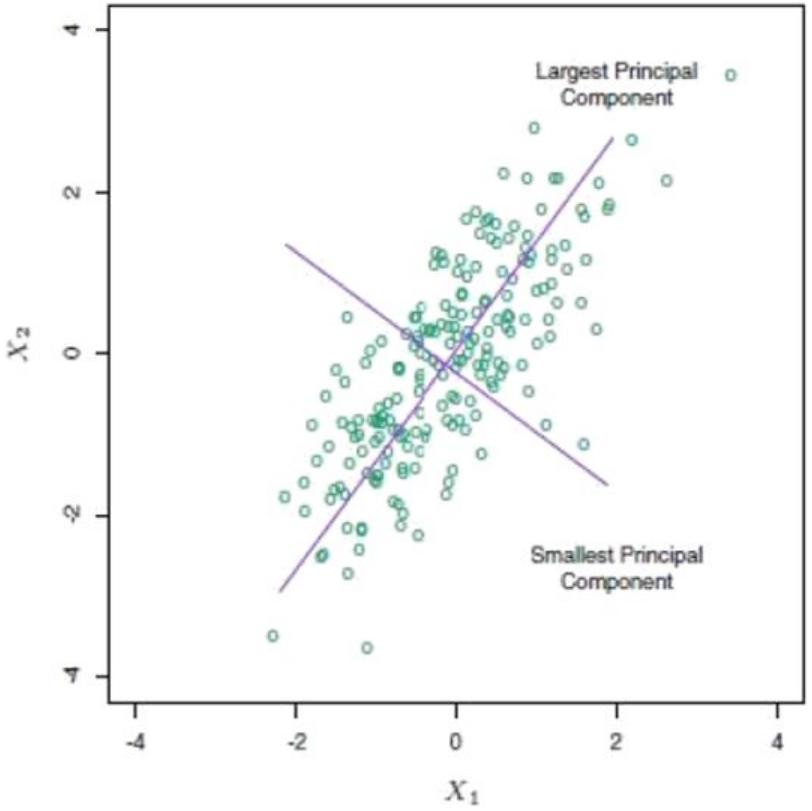
Concept of eigenvectors and eigenvalues



Copyright National University of Singapore

Principal Component Analysis Method

- The new axes are called **principal components** (PCs)
- The 1st PC accounts for the maximum variance in the data
- The 2nd PC accounts for the maximum variance that has not been accounted by the 1st variable
- The pth PC accounts for the maximum variance that has not been accounted by the p-1 variables



Copyright National University of Singapore

Principal Component Analysis Method

- The Principal Components are considered as **new variables**, and they are all **uncorrelated.**
- If a substantial amount of the total variance in the data is accounted for by a few principal components then one can use these fewer no. of variables for further investigation instead of original features.
- This subset of variables can be identified and shortlisted by looking at the **eigenvalues** (i.e. variance accounted for by the PC)

	PC1	PC2	PC3	PC4
Income	0.314	-0.145	0.676	-0.347
Education	0.237	-0.444	0.401	0.240
Age	0.484	0.135	0.004	-0.212
Residence	0.466	0.277	-0.091	0.116
Employ	0.459	0.304	-0.122	-0.017
Savings	0.404	-0.219	-0.366	0.436
Debt	-0.067	0.585	0.078	-0.281
Credit cards	-0.123	0.452	0.468	0.703

Copyright National University of Singapore

PCA EXAMPLE

Copyright National University of Singapore

PCA Example: Loan Application

Income	Education	Age	Residence	Employ	Savings	Debt	Credit cards
50000	16	28	2	2	5000	1200	2
72000	18	35	10	8	12000	5400	4
61000	18	36	6	5	15000	1000	2
88000	20	35	4	4	980	1100	4
91100	18	38	8	9	20000	0	1
45100	14	41	15	14	3900	22000	4
36200	14	29	6	5	100	7000	5
41000	12	34	9	8	5000	200	3
40000	16	32	8	7	19000	1760	2
32000	16	30	2	2	16000	550	1
29000	16	28	1	4	2100	4600	2
21240	12	26	2	2	100	10010	3
58700	12	38	9	9	4500	7800	5
41000	14	29	5	4	300	10000	6
38720	16	36	11	11	24500	540	2
88240	16	38	13	12	13600	8100	2
40000	18	39	7	6	16000	1300	2

Income: Yearly Income

Education: no. of years

Age: years

Residence: years at current address

Employ: years at current coy

Savings: current saving

Debt: current debts

Credit cards: #credit cards owned

Copyright National University of Singapore

Approach for this Example

1. **Basic Data understanding**
   - Summary statistics
   - Correlation Matrix
2. **Check for appropriateness of PCA Method**
   - Bartlett's Sphericity Test
   - Kaiser-Meyer-Olkin Test
3. **Run PCA and examine the outputs**
   - Principal Components
   - Interpretation of PC (Loadings)
   - Loading Plot
4. **How Many Components to retain?**
   - Eigenvalues (Kaiser Rule)
   - Total variance explained
   - Scree Plot
   - Communalities Matrix
5. **Proceeding to the next step**
   - Eigenvectors
   - Component Scores
   - Score Plot

Copyright National University of Singapore

1.Basic Data Understanding

Summary statistics

Correlation Matrix/Scatter Plot

Copyright National University of Singapore

Summary Statistics

Data Dimensions: (30, 8)
Missing Values found: 0
Summary Statistics:

	miss	count	mean	std	min	25%	50%	75%	max
Income	0	30.0	48646.00	18505.45	21240.0	37055.00	41100.0	58125.00	91100.0
Education	0	30.0	15.27	2.13	12.0	14.00	16.0	16.00	20.0
Age	0	30.0	32.80	4.63	26.0	29.00	33.0	36.00	41.0
Residence	0	30.0	5.93	3.98	1.0	2.25	5.0	8.00	15.0
Employ	0	30.0	5.50	3.66	1.0	2.00	4.0	8.00	14.0
Savings	0	30.0	8319.33	8550.50	0.0	1950.00	4750.0	14050.00	34000.0
Debt	0	30.0	3812.00	5042.51	0.0	800.00	1200.0	6600.00	22000.0
Credit cards	0	30.0	2.87	1.25	1.0	2.00	2.5	3.75	6.0

Copyright National University of Singapore

Correlation Matrix

	Income	Education	Age	Residence	Employ	Savings	Debt	Credit cards
Income	1.00	0.55	0.52	0.35	0.33	0.21	-0.20	-0.06
Education	0.55	1.00	0.23	0.11	0.05	0.45	-0.46	-0.30
Age	0.52	0.23	1.00	0.84	0.85	0.55	0.03	-0.13
Residence	0.35	0.11	0.84	1.00	0.95	0.57	0.19	0.05
Employ	0.33	0.05	0.85	0.95	1.00	0.54	0.25	0.02
Savings	0.21	0.45	0.55	0.57	0.54	1.00	-0.39	-0.41
Debt	-0.20	-0.46	0.03	0.19	0.25	-0.39	1.00	0.47
Credit cards	-0.06	-0.30	-0.13	0.05	0.02	-0.41	0.47	1.00

- There appears to be strong correlation between some variables.
- For example, ***Age*** is highly correlated to ***Employ*** (0.85), ***Residence*** (0.84) and ***Savings*** (0.55)

Copyright National University of Singapore

2.Check for appropriateness for PCA

Kaiser-Meyer-Olkin Test

Bartlett’s Sphericity Test

Copyright National University of Singapore

Kaiser-Meyer-Olkin Test

- The Kaiser-Meyer-Olkin (KMO) test is a statistical measure to help you assess whether your data sample is suitable for PCA.
- KMO checks how well your variables are grouped together.
  - **Are the variables correlated enough** to form meaningful components or factors?
  - **Should you even use PCA** on this dataset?
- KMO returns values between 0 and 1.
  - A higher value (closer to 1) means the data is well suited for PCA, i.e. the data is likely to form distinct and reliable components
  - A lower value (closer to 0) indicates that the data is not suitable for dimension reduction, i.e. data is too scattered or inconsistent to find clear patterns.
- A rule of thumb for interpreting the statistic:
  - KMO ≥ 0.5 indicates the sampling is acceptable
  - KMO < 0.5 indicate the sampling is NOT acceptable

https://www.ibm.com/docs/en/spss-statistics

Copyright National University of Singapore

Bartlett's Test

- Bartlett's Test of Sphericity tests the hypothesis that your correlation matrix is an identity matrix, which would indicate that your variables are unrelated and therefore unsuitable for structure detection.

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

H_0 : Correlation Matrix = Identity Matrix

H_1 : Correlation Matrix ≠ Identity Matrix

REJECT the NULL if the P-value is < 0.05

- We want to reject the Null hypothesis – i.e. we want the correlation matrix to diverge significantly from the identity matrix. If true, it implies the existence of redundancies between the variables such that PCA can summarize this with fewer variables.

https://www.ibm.com/docs/en/spss-statistics

Copyright National University of Singapore

Running the KMO and Bartlett's Test

These 2 tests provide a minimum standard which should be passed before a Principal Component analysis can be conducted successfully.

```
from factor_analyzer import FactorAnalyzer
from factor_analyzer.factor_analyzer import calculate_bartlett_sphericity

chi_square_value,p_value = calculate_bartlett_sphericity(data)
print("chi_square_value:",chi_square_value, "\nP-value:", p_value)

from factor_analyzer.factor_analyzer import calculate_kmo
kmo_all,kmo_model=calculate_kmo(data)
print("\nKaiser-Meyer-Olkin:", kmo_model)
```

```
chi_square_value: 173.01622444105612
P-value: 3.2842239591607713e-23

Kaiser-Meyer-Olkin: 0.6855653942535219
```

Copyright National University of Singapore

3.Run the Principal Component Analysis

Principal Components

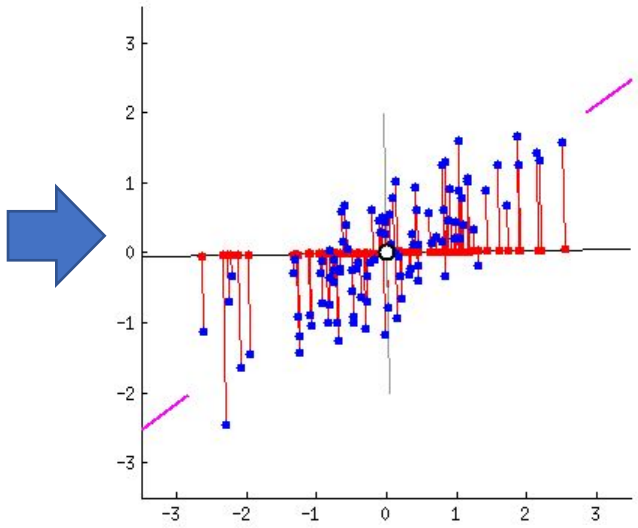
Interpretation of PC (Loading matrix)

Loading Plot

Copyright National University of Singapore

PCA Transformation

Income	Education	Age	Residence	Employ	Savings	Debt	Credit cards
50000	16	28	2	2	5000	1200	2
72000	18	35	10	8	12000	5400	4
61000	18	36	6	5	15000	1000	2
88000	20	35	4	4	980	1100	4
91100	18	38	8	9	20000	0	1
45100	14	41	15	14	3900	22000	4
36200	14	29	6	5	100	7000	5
41000	12	34	9	8	5000	200	3
40000	16	32	8	7	19000	1760	2
32000	16	30	2	2	16000	550	1
29000	16	28	1	4	2100	4600	2
21240	12	26	2	2	100	10010	3
58700	12	38	9	9	4500	7800	5
41000	14	29	5	4	300	10000	6
38720	16	36	11	11	24500	540	2
88240	16	38	13	12	13600	8100	2
40000	18	39	7	6	16000	1300	2



Principal Components

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8
Income	0.314	-0.145	0.676	-0.347	0.241	-0.494	-0.018	-0.030
Education	0.237	-0.444	0.401	0.240	-0.622	0.357	-0.103	0.057
Age	0.484	0.135	0.004	-0.212	0.175	0.487	0.657	-0.052
Residence	0.466	0.277	-0.091	0.116	0.035	0.085	-0.487	-0.662
Employ	0.459	0.304	-0.122	-0.017	0.014	0.023	-0.368	0.739
Savings	0.404	-0.219	-0.366	0.436	-0.143	-0.568	0.348	-0.017
Debt	-0.067	0.585	0.078	-0.281	-0.681	-0.245	0.196	-0.075
Credit cards	-0.123	0.452	0.468	0.703	0.195	0.022	0.158	0.058

Copyright National University of Singapore

Principal Components extracted

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8
Income	0.314	-0.145	0.676	-0.347	0.241	-0.494	-0.018	-0.030
Education	0.237	-0.444	0.401	0.240	-0.622	0.357	-0.103	0.057
Age	0.484	0.135	0.004	-0.212	0.175	0.487	0.657	-0.052
Residence	0.466	0.277	-0.091	0.116	0.035	0.085	-0.487	-0.662
Employ	0.459	0.304	-0.122	-0.017	0.014	0.023	-0.368	0.739
Savings	0.404	-0.219	-0.366	0.436	-0.143	-0.568	0.348	-0.017
Debt	-0.067	0.585	0.078	-0.281	-0.681	-0.245	0.196	-0.075
Credit cards	-0.123	0.452	0.468	0.703	0.195	0.022	0.158	0.058

PC1 = 0.314*Income + 0.237*Education + 0.484*Age + 0.466*Residence +
0.459*Employ + 0.404*Savings + -0.67*Debt + -0.123*Credit cards

Copyright National University of Singapore

Interpreting Principal Components

- Principal Components can be interpreted to see how much each original variable contributes to the construction of each PC.
- We can compute this correlation by taking the square root of the eigenvalues and multiply them by the corresponding eigenvectors to obtain PC **Loadings**:

```
# get eigenvalues to calculate the loadings
eigenvalues = pca.explained_variance_
print('Loading Matrix:');
loadings= eigenvectors * np.sqrt(eigenvalues)
```

- The result is the loading matrix, which tells you how strongly each original variable influences each principal component.

Copyright National University of Singapore

Component Loading Matrix

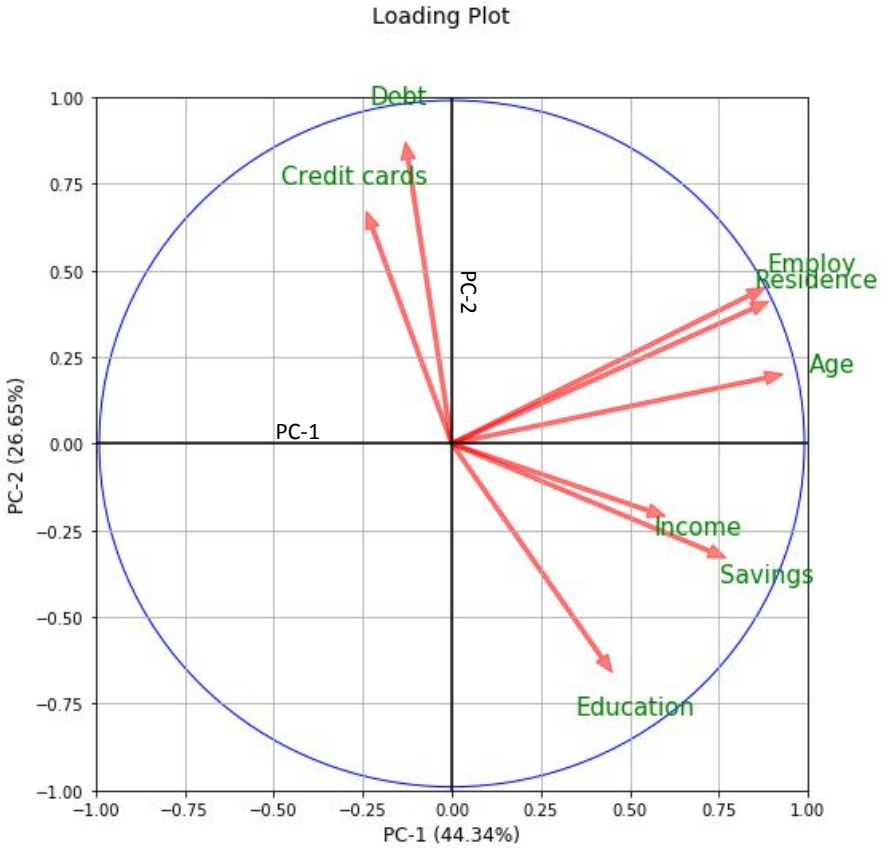
	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8
Income	0.6015	-0.2153	0.7028	-0.2573	0.1572	-0.2050	-0.0065	-0.0062
Education	0.4540	-0.6594	0.4169	0.1780	-0.4057	0.1482	-0.0371	0.0118
Age	0.9272	0.2005	0.0042	-0.1572	0.1141	0.2021	0.2366	-0.0107
Residence	0.8927	0.4114	-0.0946	0.0860	0.0228	0.0353	-0.1754	-0.1366
Employ	0.8793	0.4515	-0.1268	-0.0126	0.0091	0.0095	-0.1325	0.1525
Savings	0.7739	-0.3252	-0.3805	0.3233	-0.0933	-0.2357	0.1253	-0.0035
Debt	-0.1284	0.8688	0.0811	-0.2084	-0.4442	-0.1017	0.0706	-0.0155
Credit cards	-0.2356	0.6713	0.4865	0.5213	0.1272	0.0091	0.0569	0.0120

Rename the components:
PC1: “Financial Stability”
PC2: “Liabilities”
PC3: “Income”

Component ***loadings*** are interpreted as ***correlation*** coefficients between the original variables and components. It is analogous to *Pearson's r*, and the squared *loading* is the percent of variance in that variable explained by the component.

Copyright National University of Singapore

Loading Plot



- The Loading Plot graphs the **loadings** of Principal components.
- Loading Plots allow you to identify which variables have the **largest effect** on each component.
- Loadings close to -1 or 1 indicate that the variable **strongly influences** the component.
- Loadings close to 0 indicate that the variable has a **weak influence** on the component.
- Evaluating the loadings can also help you **characterize each component** in terms of the variables.

Copyright National University of Singapore

4.How Many Components to retain?

Eigenvalues (Kaiser Rule)

Total variance explained

Scree Plot

Communalities Matrix

Copyright National University of Singapore

What are eigenvalues?

- **Eigenvalue** represent the total amount of **variance that is explained** by a given principal component. Eigenvalues are standardized and hence it carries one unit of information.

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8
Income	0.6015	-0.2153	0.7028	-0.2573	0.1572	-0.2050	-0.0065	-0.0062
Education	0.4540	-0.6594	0.4169	0.1780	-0.4057	0.1482	-0.0371	0.0118
Age	0.9272	0.2005	0.0042	-0.1572	0.1141	0.2021	0.2366	-0.0107
Residence	0.8927	0.4114	-0.0946	0.0860	0.0228	0.0353	-0.1754	-0.1366
Employ	0.8793	0.4515	-0.1268	-0.0126	0.0091	0.0095	-0.1325	0.1525
Savings	0.7739	-0.3252	-0.3805	0.3233	-0.0933	-0.2357	0.1253	-0.0035
Debt	-0.1284	0.8688	0.0811	-0.2084	-0.4442	-0.1017	0.0706	-0.0155
Credit cards	-0.2356	0.6713	0.4865	0.5213	0.1272	0.0091	0.0569	0.0120

```
Eigenvalues      : [3.67  2.206 1.081 0.55  0.425 0.172 0.13  0.043]
```

Eigenvalues are **the sum of squared** component loadings across all items.

eigenvalues can be extracted using *explained_variance_*

Copyright National University of Singapore

The Kaiser Rule

- The **Kaiser Rule** is perhaps the most widely used criterion for selecting which components to keep.
- It is based on the idea that a component should be considered insignificant if it does worse than a single field.
- Each single field contains one unit of standardized variance, thus components with eigenvalues below 1 are not extracted.

```
                 PC1  PC2    PC3    PC4    PC5    PC6    PC7    PC8
Eigenvalues    : [3.67  2.206 1.081 0.55   0.425  0.172  0.13   0.043]
```

How many components should be retained?

Copyright National University of Singapore

Total Variance Explained

- Eigenvalues can also be expressed in terms of a **percentage of the total variance** of the original fields.
- The second row of the table denotes the proportion of the variance attributable to each component, and the next row denotes the proportion of the **variance jointly explained** by all components up to that point.
- The first 4 components explains 90.69% of the variability with ~10% loss of information.

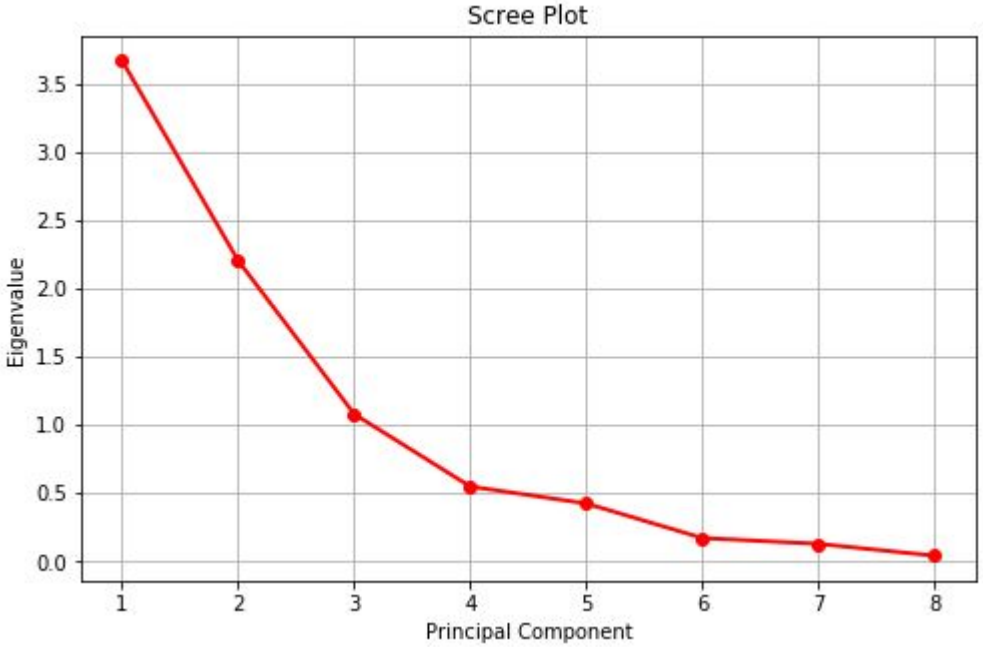
	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8
Eigenvalues :	3.67	2.206	1.081	0.55	0.425	0.172	0.13	0.043
% Variance Explained:	44.34	26.65	13.06	6.64	5.14	2.08	1.57	0.51
% Cumulative) :	44.34	70.99	84.05	90.69	95.83	97.91	99.48	99.99

Copyright National University of Singapore

Scree Plot

The Scree Plot shows the eigenvalues vs the components in decreasing order:

```
Eigenvalues          : [3.67   2.206 1.081 0.55   0.425 0.172]
% Variance Explained: [44.345 26.65   13.059   6.644   5.14    2.081]
% Cumulative)        : [44.345 70.995 84.054 90.698 95.838 97.919]
```



Scree Plot test

Look for a large drop, followed by a "plateau" in the eigenvalues – i.e. a bend or elbow. This indicates a transition from the large to smaller values. Beyond the elbow point, the variances explained tapers off.

If the scree is "not ideal" – i.e. you don't see any clear bends, then fall back on previous 2 criteria.

Copyright National University of Singapore

Communalities

** Communalities :

	2	3	4
Income	0.408208	0.902090	0.968296
Education	0.640920	0.814708	0.846378
Age	0.899891	0.899908	0.924620
Residence	0.966167	0.975117	0.982515
Employ	0.977002	0.993088	0.993247
Savings	0.704765	0.849539	0.954061
Debt	0.771255	0.777831	0.821247
Credit cards	0.506117	0.742829	1.014565

Does the solution account for all the original variables?

- Communality represents the total amount of variance of a specific field that is jointly accounted for by all the components. It is calculated as the sum of squared loadings of the field across all components.
- **High communality** values indicate that the original field is **sufficiently explained** by the reduced PCA solution.
- **Low communality** implies an **insignificant** contribution to the formation of the PCA Solution
- communalities > 0.5 are desired

Copyright National University of Singapore

5.Proceeding to the next step

Copyright National University of Singapore

Eigenvectors

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8
Income	0.314	-0.145	0.676	-0.347	0.241	-0.494	-0.018	-0.030
Education	0.237	-0.444	0.401	0.240	-0.622	0.357	-0.103	0.057
Age	0.484	0.135	0.004	-0.212	0.175	0.487	0.657	-0.052
Residence	0.466	0.277	-0.091	0.116	0.035	0.085	-0.487	-0.662
Employ	0.459	0.304	-0.122	-0.017	0.014	0.023	-0.368	0.739
Savings	0.404	-0.219	-0.366	0.436	-0.143	-0.568	0.348	-0.017
Debt	-0.067	0.585	0.078	-0.281	-0.681	-0.245	0.196	-0.075
Credit cards	-0.123	0.452	0.468	0.703	0.195	0.022	0.158	0.058

Principal components are the linear combinations of the original variables that account for the variance in the data.

The **eigenvectors**, which are comprised of **coefficients** corresponding to each variable, are used to calculate the principal **component scores**.

The coefficients indicate the relative weight of each variable in the component.

```
PC1 = 0.314*Income    + 0.237*Education + 0.484*Age +
      0.466*Residence + 0.459*Employ    + 0.404*Savings +
      -0.67*Debt      + -0.123*Credit cards
```

`pca.components_`

Copyright National University of Singapore

Component Scores

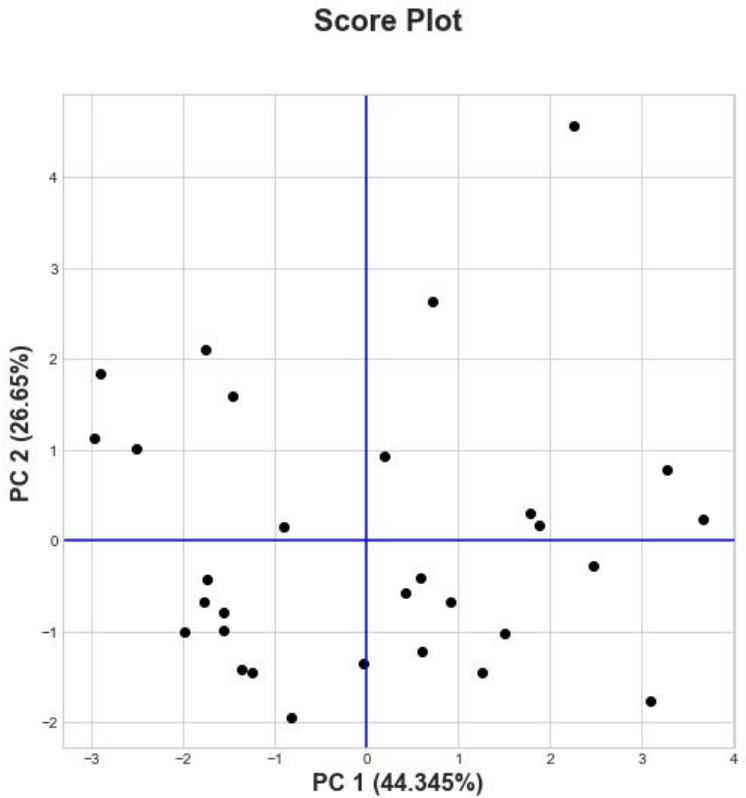
	PC1	PC2	PC3	PC4
0	-1.356145	-1.422152	0.169596	-0.337685
1	1.790714	0.306738	1.508913	0.621809
2	1.252730	-1.443638	0.334810	-0.056943
3	0.597244	-1.226393	3.172501	0.059572
4	3.086463	-1.759539	0.659380	-0.939320
5	2.258250	4.576150	0.046110	-0.843019
6	-1.469670	1.590170	0.526923	0.885788
7	0.186354	0.925404	-0.924450	-0.096049
8	0.913751	-0.676040	-1.107618	0.514616
9	-0.816324	-1.949886	-1.367173	-0.051855

- These **component scores** are the values of the new "features".
- They can be used like any other features in subsequent tasks.
- Note that the scores are **standardized values** with mean=0 and SD=1.
- Hence the scores represent the number of standard deviations above or below the overall mean where each customer lies.

Customer #4: Very good FINANCIAL STABILITY (PC1), low on LIABILITIES (PC2) and average INCOME (PC3)

Copyright National University of Singapore

Score Plot



- This score plot graphs the scores of the First PC versus the scores of the Second PC.
- If the first two components account for most of the variance in the data, you can use the score plot to assess the data structure and detect clusters, outliers, and trends.
- **Clusters**: Observations that are close to each other on the plot have similar profiles and are likely more similar to each other in terms of the original variables.
- **Separation**: A clear separation between clusters or groups in the plot indicates that the principal components effectively capture differences between these groups.
- **Outliers:** Observations that are far from the main cluster of points can be considered outliers.

Copyright National University of Singapore

Some Points to note about PCA

- **Sampling adequacy**: ideally, there should be 150+ cases and there should be ratio of at least five cases for each variable (Pallant, 2010)
  - You can make use of the Kaiser-Meyer-Olkin (KMO) test as a guideline
- **Correlations exist (Suitability):** there should be some linear correlation among the factors to be considered for PCA
  - PCA assumes a linear relationship between features
  - You can make use of the Bartlett's Sphericity Test as a guideline
- **Continuous variables:** variables should be non-categorical
  - PCA doesn't work with categorical variable. Too many one-hot-encoding will result in extremely sparse and information-parched multidimensional space that and PCA cannot perform well on, since several features contain only two unique values.
- **Outliers:** PCA is sensitive to outliers
  - Remove data that are > 3 standard deviations
  - Standardize your data

Copyright National University of Singapore

What is meant by data loss?

- The goal of PCA is to extract the smallest number of components which **account for as much as possible of the information** of the original features.
- Hence, there will be some amount of information that will be lost.
- To illustrate information lost, we will **reconstruct** the original data using only the saved principal components.

```
[[ 19.   4.  90. 150.]
 [ 43.  12.  30.  35.]
 [ 13.   3.  10.  20.]
 [ 60.  14. 100.  80.]
 [  5.   1.  30.  55.]
 [ 56.  11.  25.  35.]
 [ 25.   7.  30.  28.]
 [  3.   1.  65.  82.]
 [ 40.   9.  15.  30.]
 [ 65.  15.  20.  40.]]
```

Original Data

```
[[ 23.   5. 106. 138.]
 [ 35.   8.  27.  36.]
 [ 37.   9.  14.  20.]
 [ 28.   6.  73.  97.]
 [ 33.   8.  39.  52.]
 [ 36.   8.  24.  33.]
 [ 35.   8.  25.  34.]
 [ 29.   7.  65.  86.]
 [ 36.   9.  19.  26.]
 [ 36.   8.  24.  33.]]
```

1 PC (76.994%)

```
[[ 14.   3. 103. 139.]
 [ 44.  10.  29.  36.]
 [ 14.   4.   7.  22.]
 [ 66.  15.  85.  92.]
 [  5.   1.  30.  55.]
 [ 54.  12.  30.  31.]
 [ 28.   7.  22.  34.]
 [  6.   1.  58.  88.]
 [ 38.   9.  20.  26.]
 [ 61.  14.  32.  30.]]
```

2 PCs (95.441%)

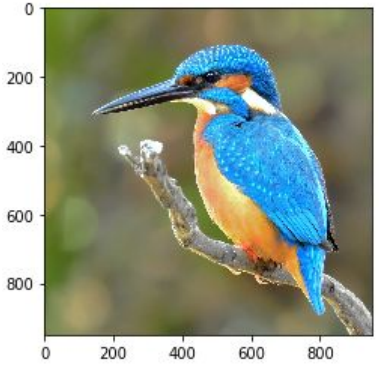
```
[[ 19.   3.  90. 149.]
 [ 43.  10.  30.  34.]
 [ 12.   3.   9.  20.]
 [ 59.  14.  99.  80.]
 [  4.   1.  29.  55.]
 [ 55.  12.  24.  35.]
 [ 25.   6.  30.  27.]
 [  2.   1.  64.  82.]
 [ 39.   9.  14.  30.]
 [ 65.  14.  20.  39.]]
```

3 PCs (99.973%)

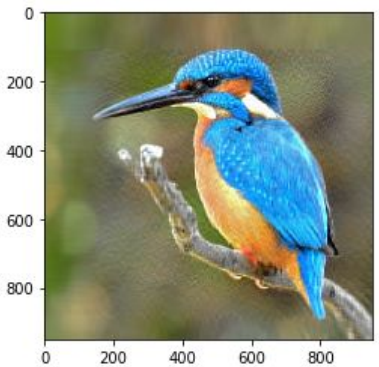
pca.inverse_transform()

Copyright National University of Singapore

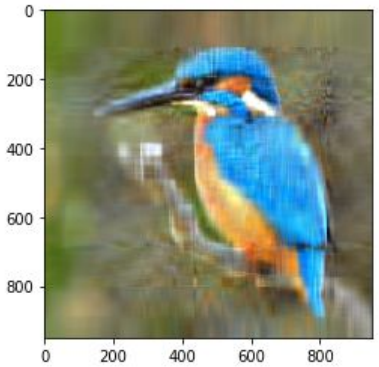
Data loss in Image Compression



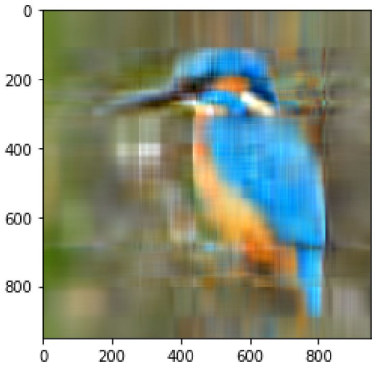
100%, 3800



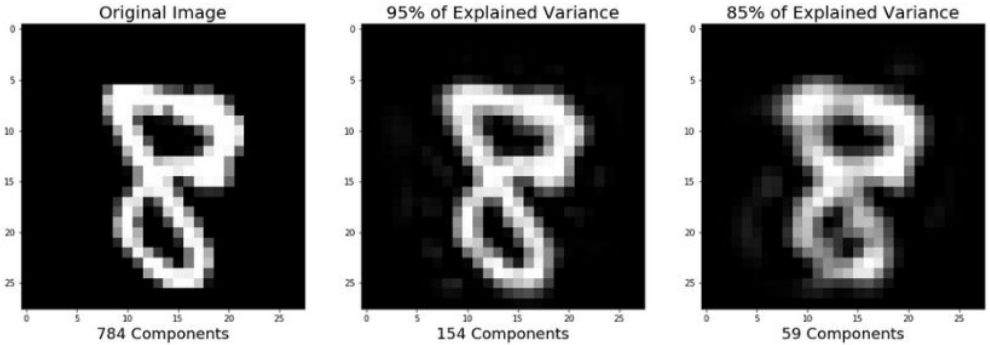
95%, 60 PC



85%, 18 PC



75%, 9 PC



Copyright National University of Singapore

Principal Component Analysis Using Python

Since we are not doing PCA from scratch, the process of performing is as follows (like the example we did earlier):

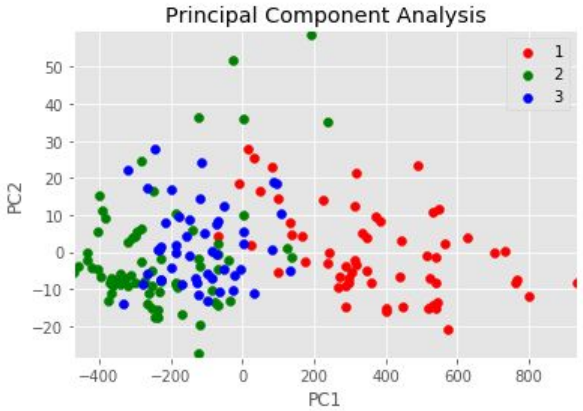
1. Read in the dataset
2. Basic Data Understanding and Preparation
3. Check appropriateness of data for PCA
4. Call the PCA algorithm (from sklearn.decomposition import PCA)
5. Plot the results
6. Decide on the number of components to retain
7. Proceeding to the next step – Predictive Modeling or Visualisation

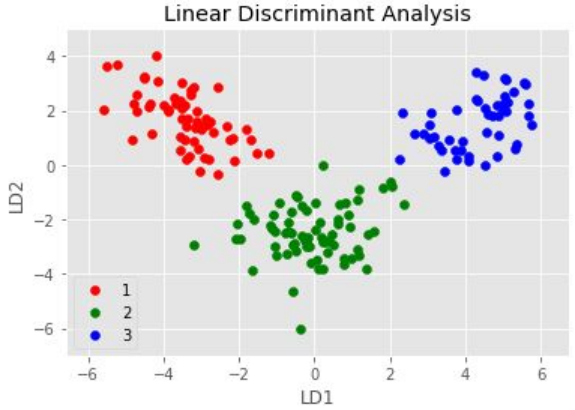
Copyright National University of Singapore

LINEAR DISCRIMINANT ANALYSIS (LDA)

Copyright National University of Singapore

What if you had access to class labels?





We not only reduce the number of features BUT at the same time create class separability. This is most useful for multi-variate classification-type problems.

Copyright National University of Singapore

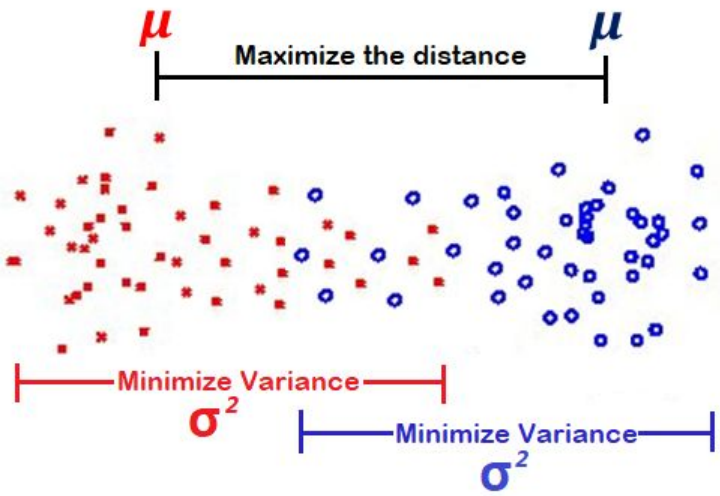
Linear Discriminant Analysis

- Originally developed in 1936 by R.A. Fisher, Discriminant Analysis is a classic method of classification.
- There are lots of similarities between LDA and PCA.
- Whereas PCA yields the directions (principal components) that maximize the variance of the data, LDA also aims to find the directions that **maximize the separation** (or discrimination) between different classes, which can be useful in pattern classification problem.
- LDA takes the **class label** into consideration whereas PCA ignores them.
- Since it uses the class information, LDA is a "**supervised learning**" method while PCA is an unsupervised learning method.

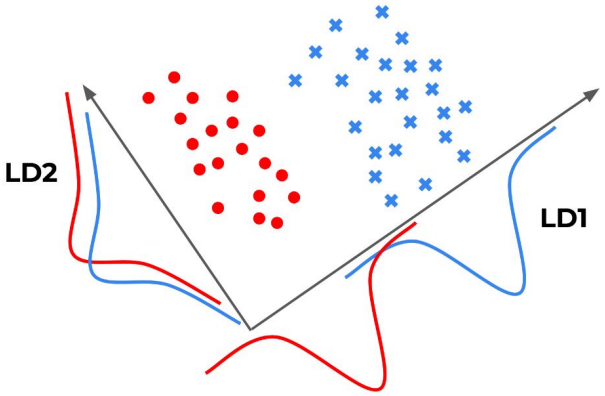
Copyright National University of Singapore

The LDA Method

- LDA finds a new dimension that yields
  - Maximum separation between the class means
  - Minimum variance within class



$$\frac{(\mu - \mu)}{\sigma^2 + \sigma^2}$$



Copyright National University of Singapore

The LDA Method (cont'd)

- This idea behind LDA can be illustrated using two figures below. The data contains two classes, shown in red and black. Similar to PCA, the data points are being projected onto a new dimensional line.
- Assume that there are two choices for the dimensional line. The goal of LDA is to find a projection where the projected data from two classes **do not overlap** as far as possible. Therefore, the projection in Figure 2 is preferred.

Figure 1

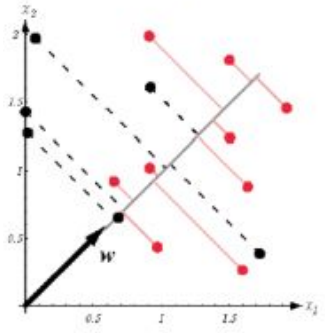
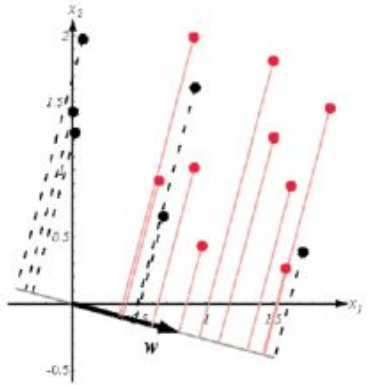
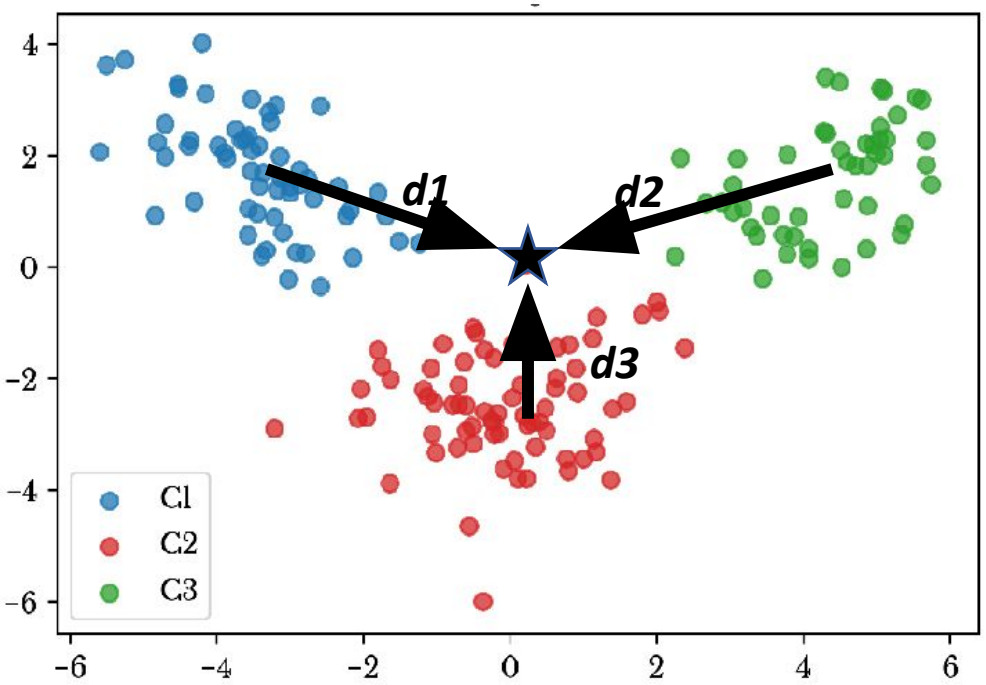


Figure 2



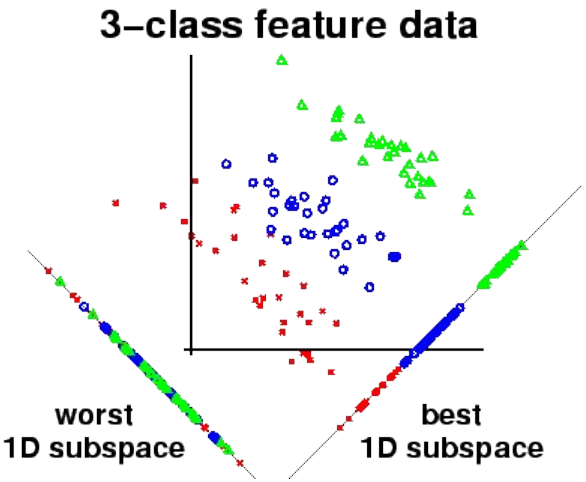
Copyright National University of Singapore

LDA method for more than 2 variables



★ = *Mean value of the projected data*

$$\frac{d1 + d2 + d3}{\sigma^2 + \sigma^2 + \sigma^2}$$



Copyright National University of Singapore

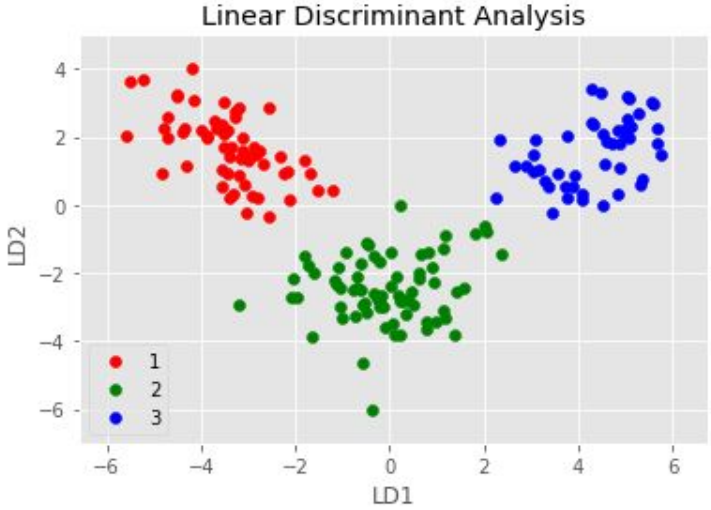
Example

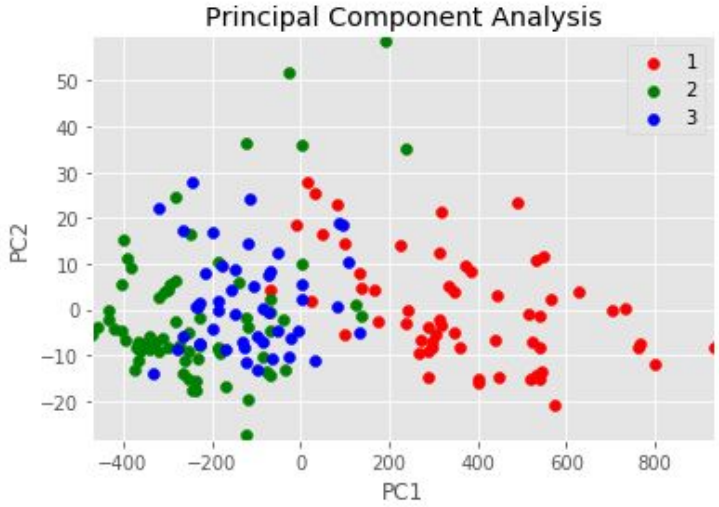
	alcohol	malic_acid	ash	ty_of_ash	agnesium	l_phenols	flavanoids	d_phenols	hocyanins	_intensity	hue	ted_wines	proline
0	14.23	1.71	2.43	15.6	127	2.8	3.06	0.28	2.29	5.64	1.04	3.92	1065
1	13.2	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.4	1050
2	13.16	2.36	2.67	18.6	101	2.8	3.24	0.3	2.81	5.68	1.03	3.17	1185
3	14.37	1.95	2.5	16.8	113	3.85	3.49	0.24	2.18	7.8	0.86	3.45	1480
4	13.24	2.59	2.87	21	118	2.8	2.69	0.39	1.82	4.32	1.04	2.93	735
5	14.2	1.76	2.45	15.2	112	3.27	3.39	0.34	1.97	6.75	1.05	2.85	1450
6	14.39	1.87	2.45	14.6	96	2.5	2.52	0.3	1.98	5.25	1.02	3.58	1290
7	14.06	2.15	2.61	17.6	121	2.6	2.51	0.31	1.25	5.05	1.06	3.58	1295
8	14.83	1.64	2.17	14	97	2.8	2.98	0.29	1.98	5.2	1.08	2.85	1045
9	13.86	1.35	2.27	16	98	2.98	3.15	0.22	1.85	7.22	1.01	3.55	1045
10	14.1	2.16	2.3	18	105	2.95	3.32	0.22	2.38	5.75	1.25	3.17	1510
11	14.12	1.48	2.32	16.8	95	2.2	2.43	0.26	1.57	5	1.17	2.82	1280
12	13.75	1.73	2.41	16	89	2.6	2.76	0.29	1.81	5.6	1.15	2.9	1320
13	14.75	1.73	2.39	11.4	91	3.1	3.69	0.43	2.81	5.4	1.25	2.73	1150
14	14.38	1.87	2.38	12	102	3.3	3.64	0.29	2.96	7.5	1.2	3	1547
15	13.63	1.81	2.7	17.2	112	2.85	2.91	0.3	1.46	7.3	1.28	2.88	1310
16	14.3	1.92	2.72	20	120	2.8	3.14	0.33	1.97	6.2	1.07	2.65	1280
17	13.83	1.57	2.62	20	115	2.95	3.4	0.4	1.72	6.6	1.13	2.57	1130

This data set has 178 examples from three classes. Each example consists of 13 real-valued features.

Copyright National University of Singapore

Results of LDA vs PCA





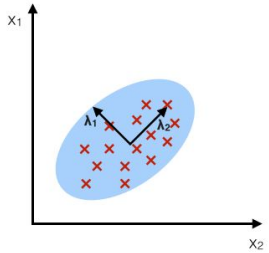

```
LDA Explained Variance [0.6875 0.3125]
PCA Exlained Variance: [0.9981 0.0017]
```

Copyright National University of Singapore

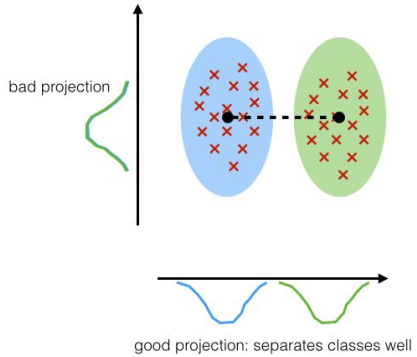
PCA vs LDA

- PCA maximizes the variation explained in the data
- LDA maximizes the separation of the classes in the data
- Both rank the components according to their importance:
  - PCA ranks them according to the largest amount of variance explained in the data
  - LDA ranks them according to the largest amount of variance explained between the classes

PCA:
component axes that maximize the variance



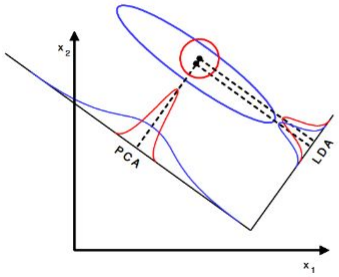
LDA:
maximizing the component axes for class-separation

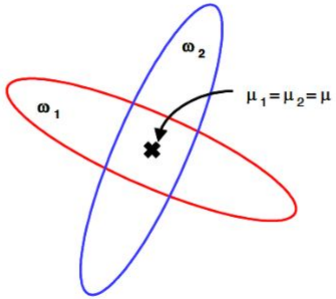


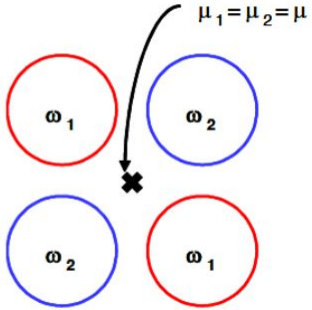
Copyright National University of Singapore

Some Assumptions for LDA

- Assumes that the data is uni-modal gaussian (normally distributed)
- LDA will fail if the discriminatory information is not in the mean but in the variance of the data







- LDA is not stable for problems with small sample size
- LDA produces c-1 discriminant vectors, where c = number of classes

Copyright National University of Singapore

Demo, Exercises and Workshop

Copyright National University of Singapore

Thank You!

Copyright National University of Singapore